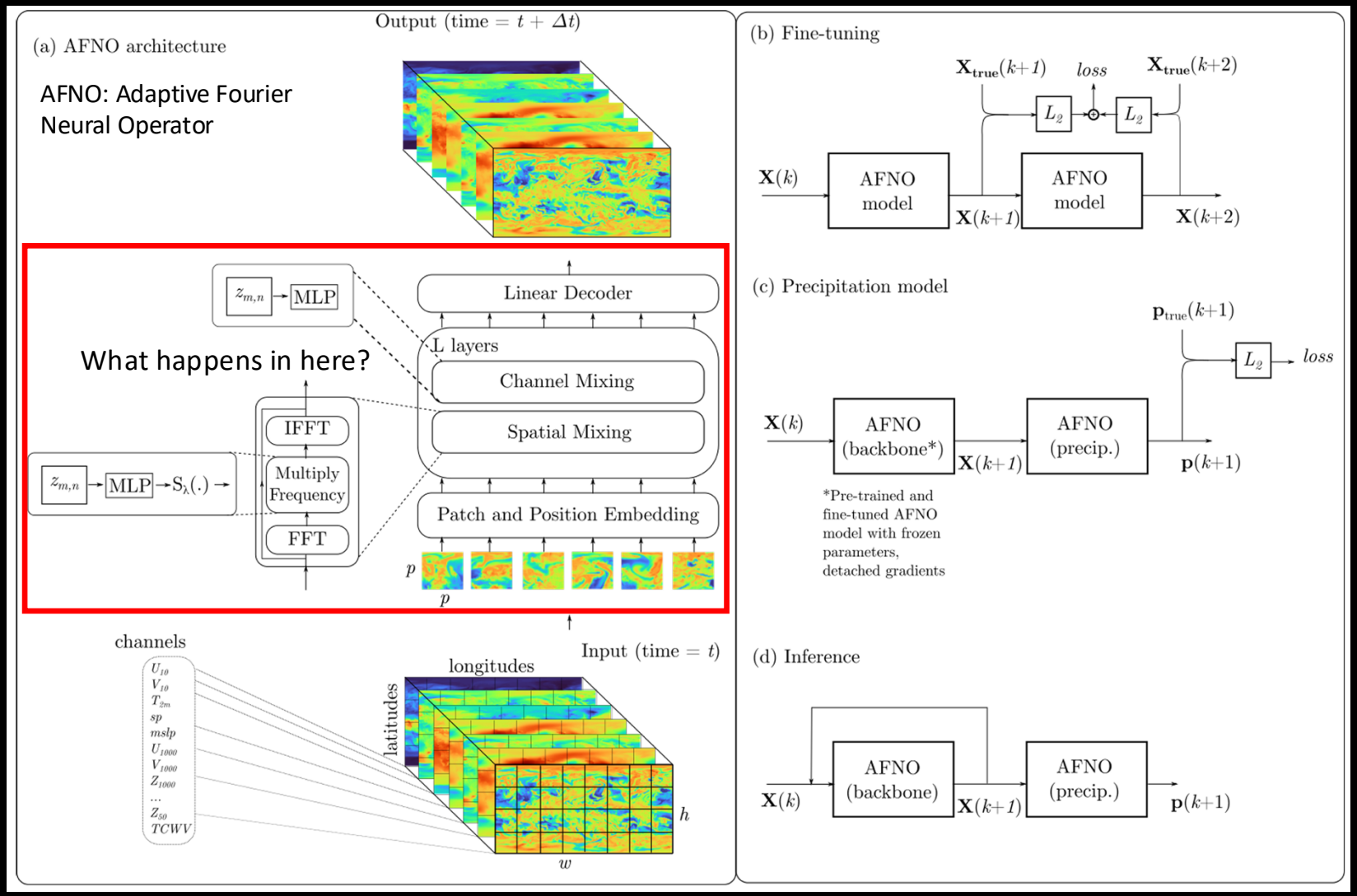
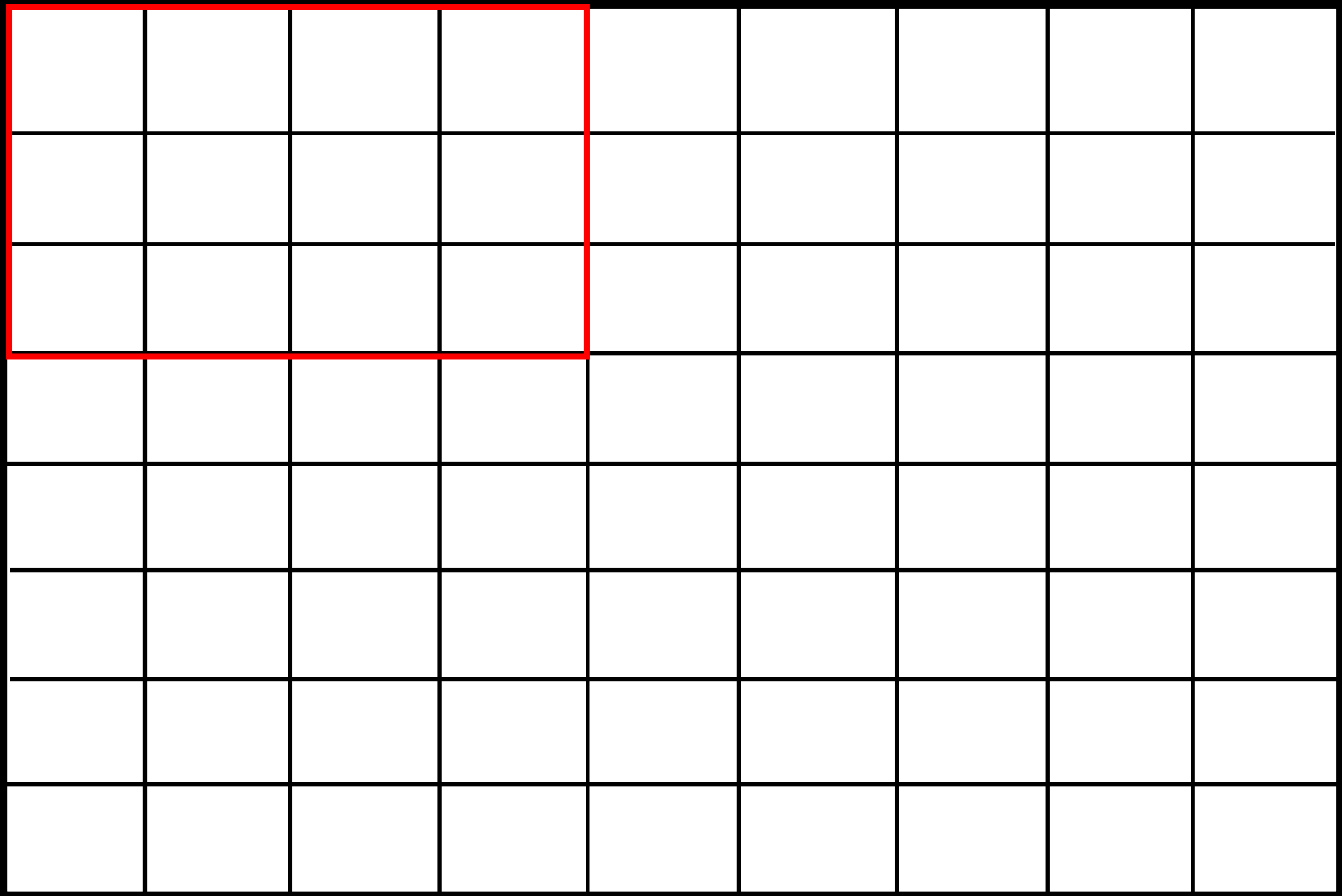
Module 3: Modern ML Model Architectures

Adaptive Fourier Neural Operators

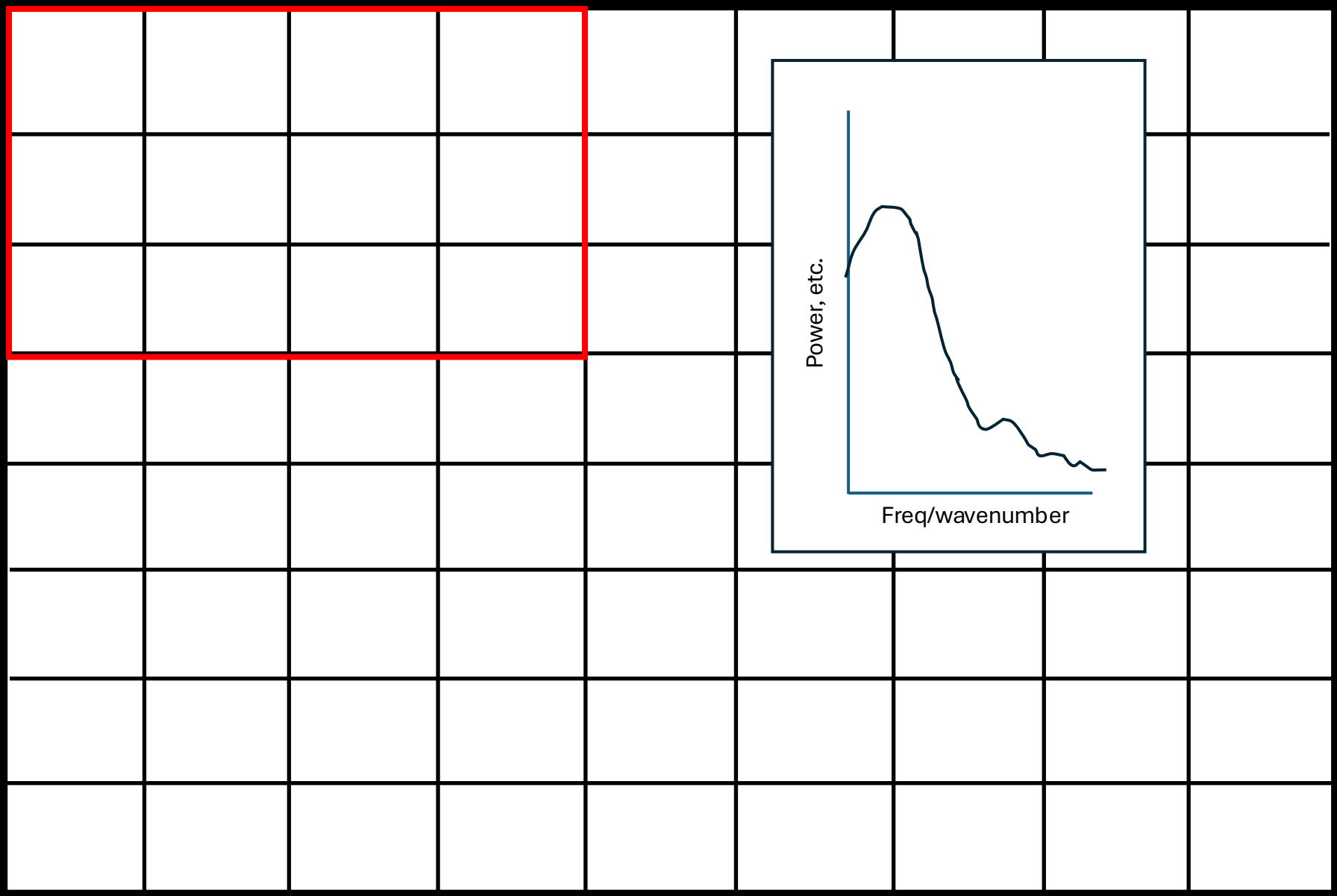
Nvidia FourCastNet (early versions c. 2022)



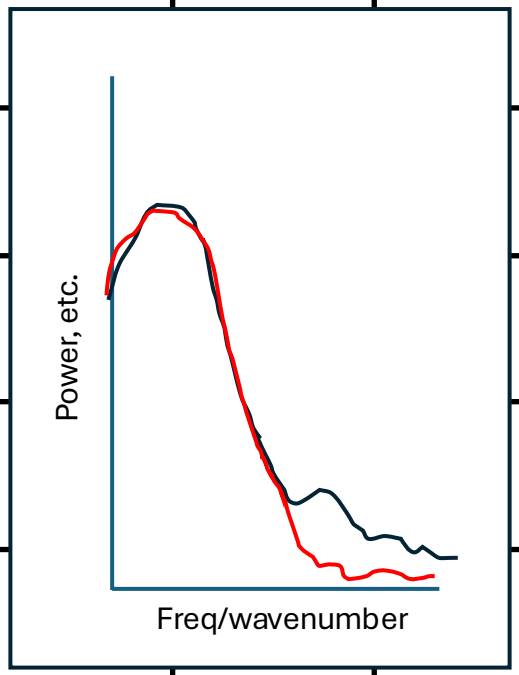
1. Break up the domain into small patches and send geo-encoding sequence of patches into ANFO layer.



2. Do 2D discrete FFT to transform input tensor to spectral space.



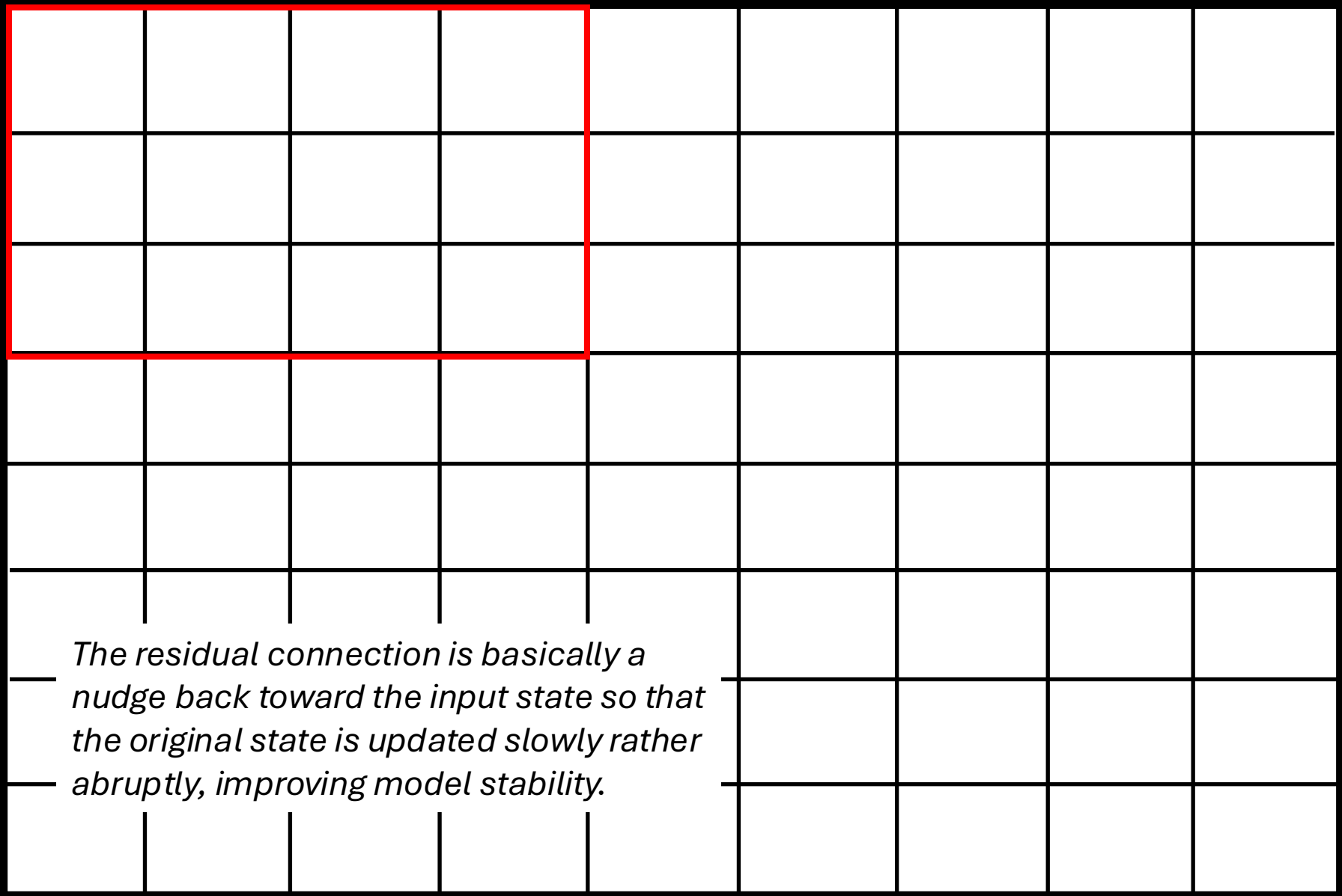
3. Weighting and soft-thresholding (sparsification).



This is the neural network part, so the architecture learns linear operators in spectral space, mixing Fourier modes and passing information between groups of nearby frequencies/wavenumbers.

Sparsification is promoted by damping of small Fourier coefficients.

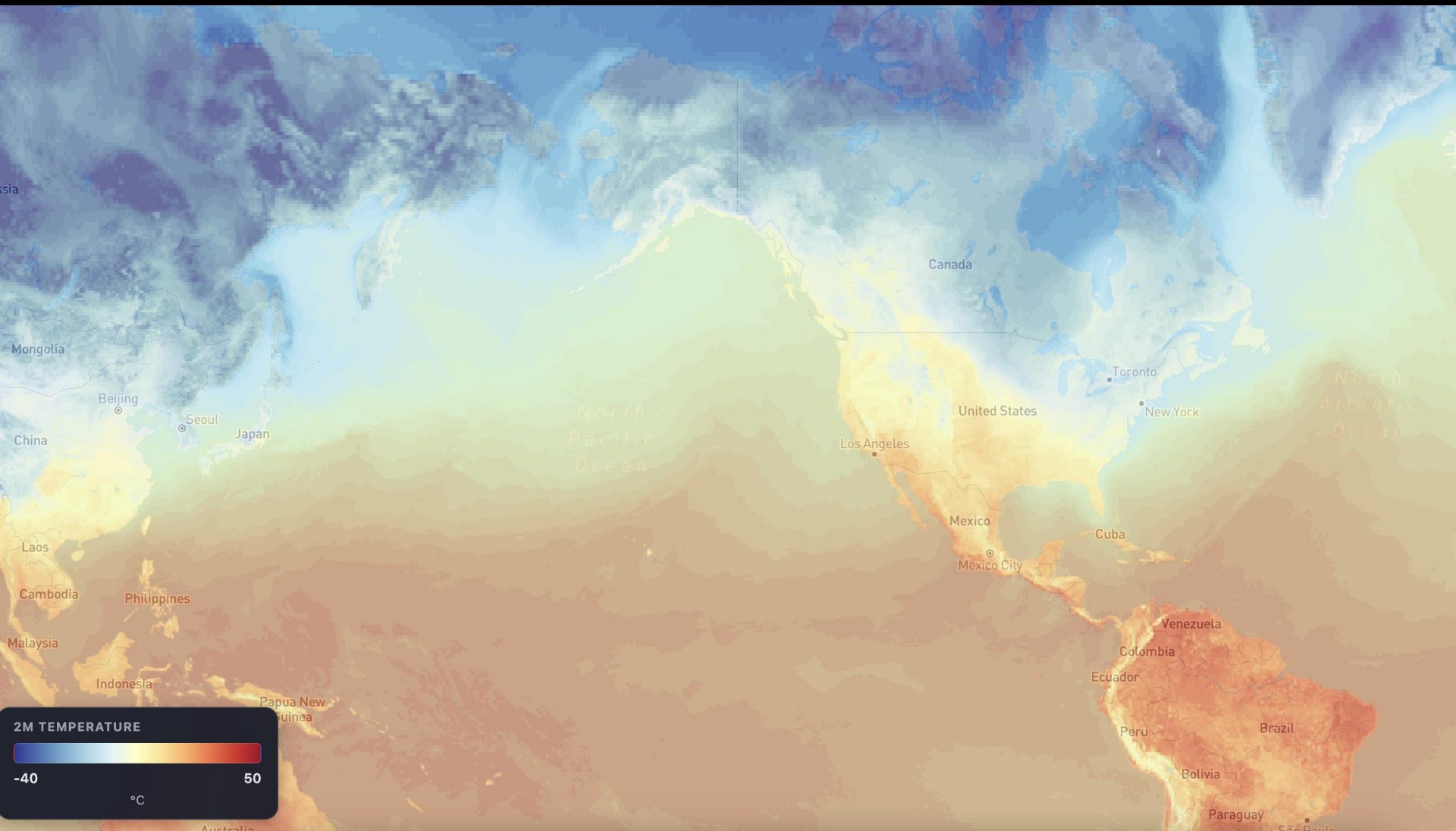
4. Convert back to physical space (like the input variables) and add residual connection.



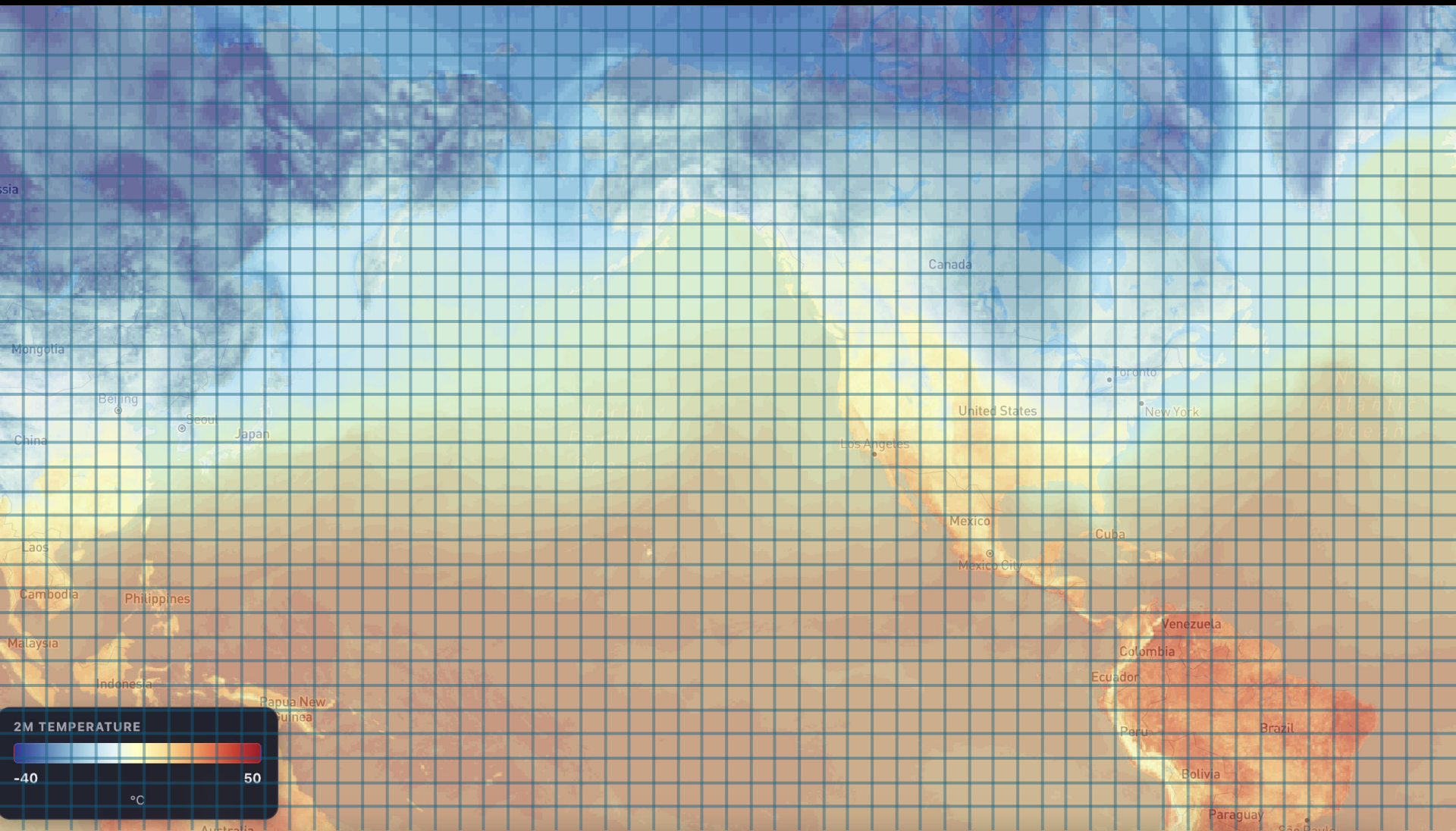
The residual connection is basically a nudge back toward the input state so that the original state is updated slowly rather abruptly, improving model stability.

Transformers

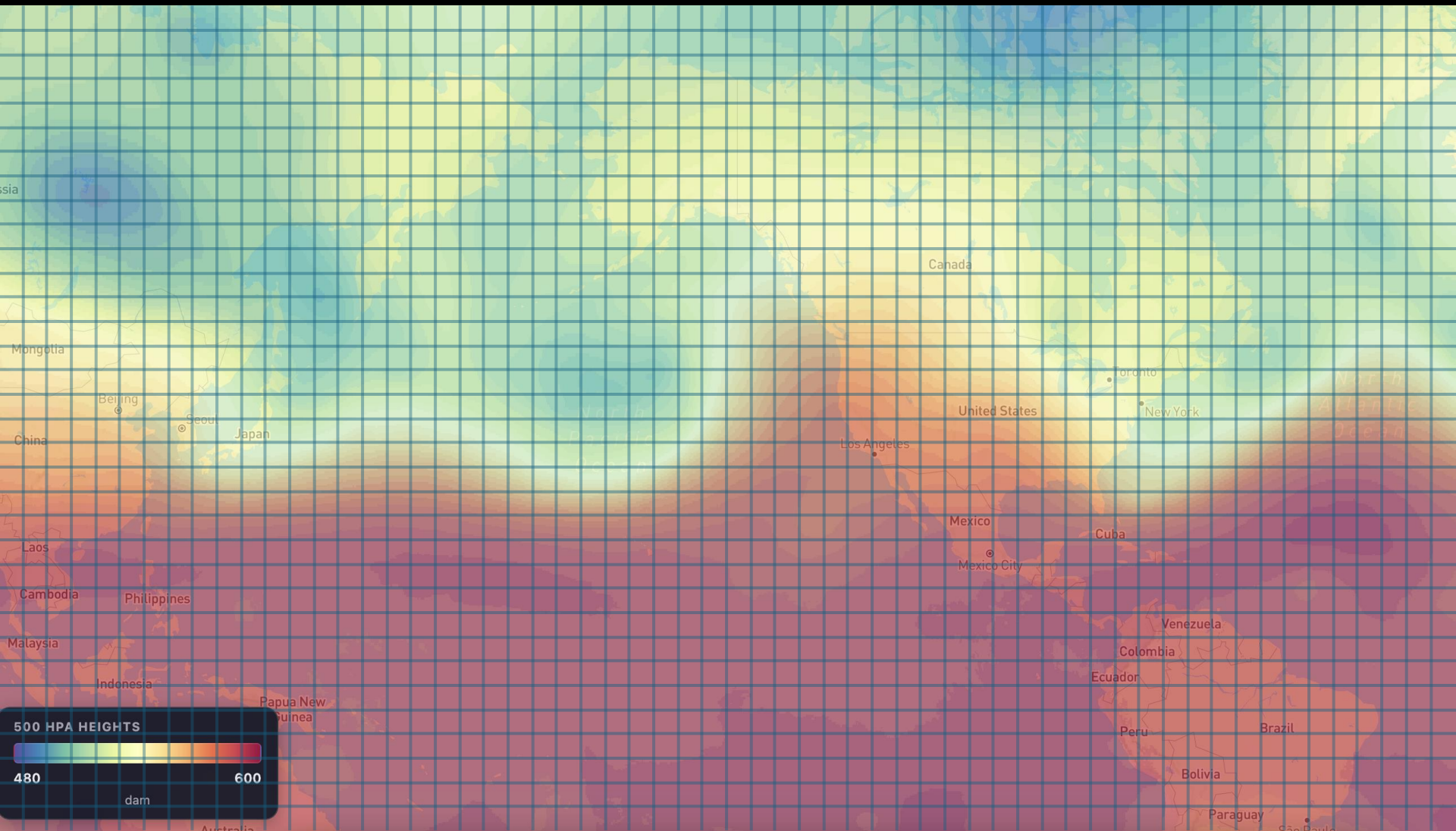
Transformers are able to efficiently pass information globally, enabling conditional "learning" of large-scale interactions in the atmosphere.



Transformers pass messages between tokens (blue squares). Each token contains several grid points.

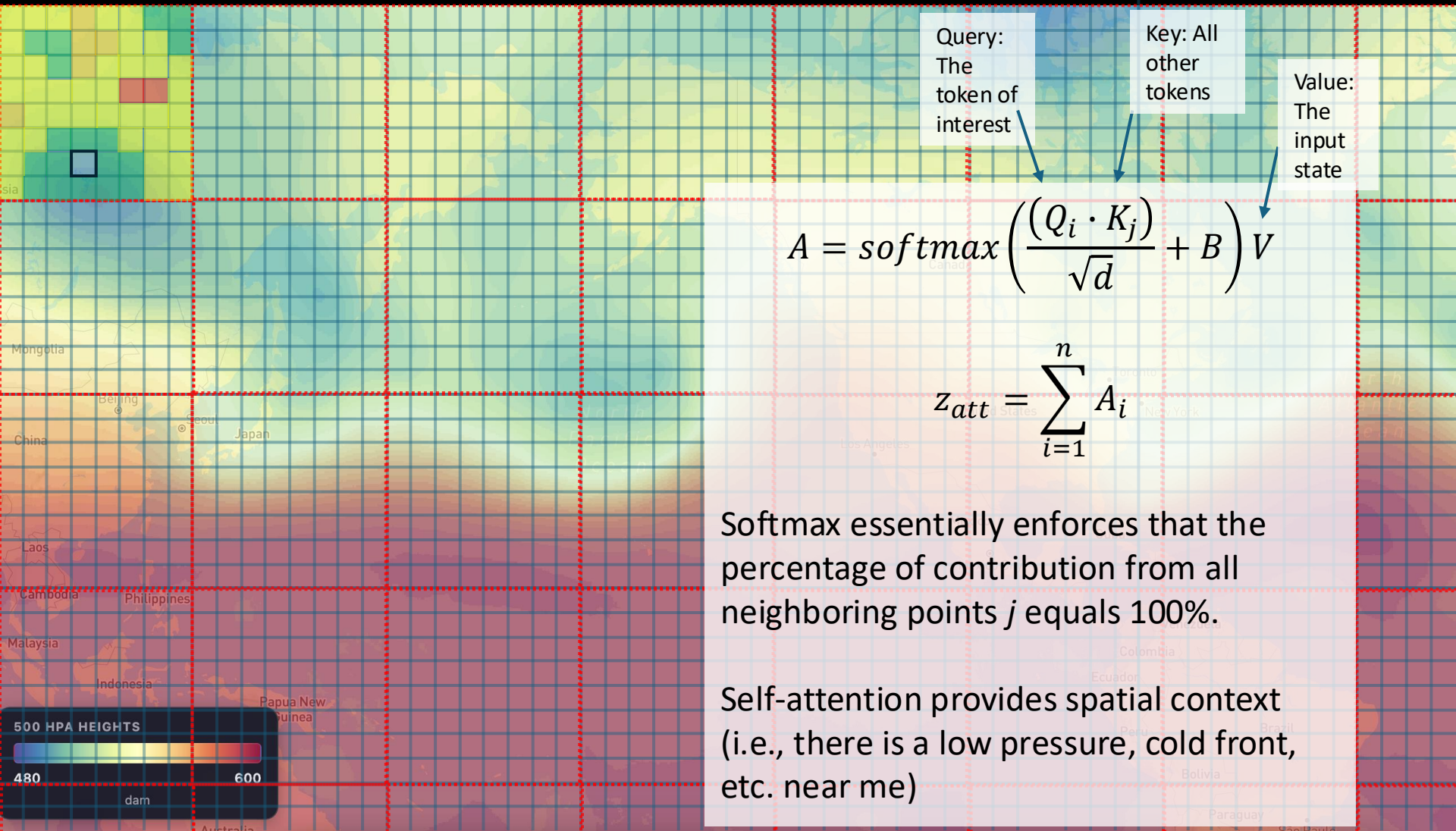


Multiple variables can be combined together into the same token, which is then flattened into a 1D vector that contains the original information but without physical meaning.



Self-attention is executed within windows (red dashed squares).

How much do each of the neighboring tokens in my window influence me?



Query: The token of interest

Key: All other tokens

Value: The input state

$$A = softmax\left(\frac{(Q_i \cdot K_j)}{\sqrt{d}} + B\right)V$$

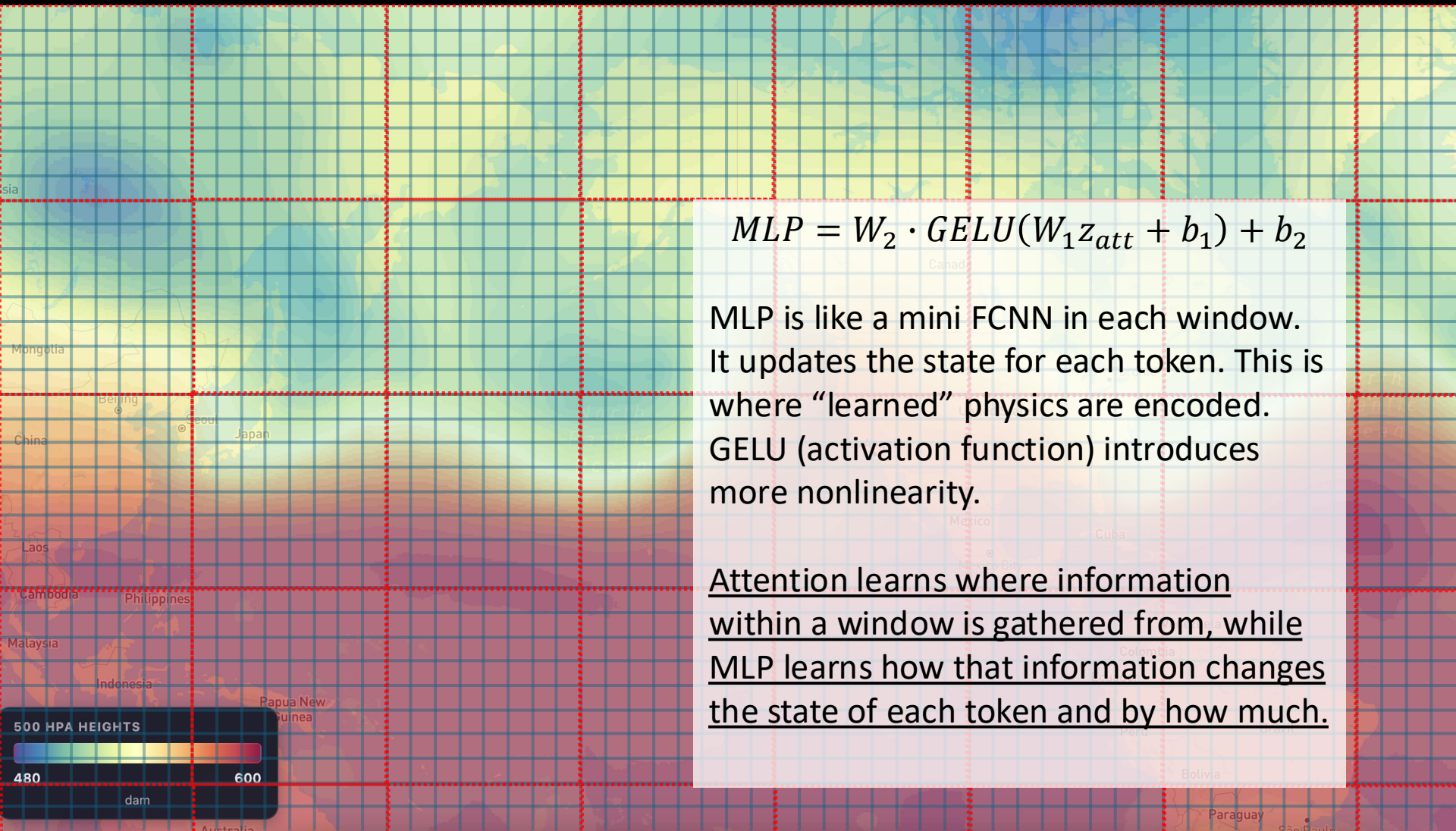
$$z_{att} = \sum_{i=1}^{n} A_i$$

Softmax essentially enforces that the percentage of contribution from all neighboring points *j* equals 100%.

Self-attention provides spatial context (i.e., there is a low pressure, cold front, etc. near me)

Next, apply learned weights and biases (MLP, or multi-layer perceptron, stage).

Given the relative importance of surrounding tokens, what is the direction and magnitude of the correction?



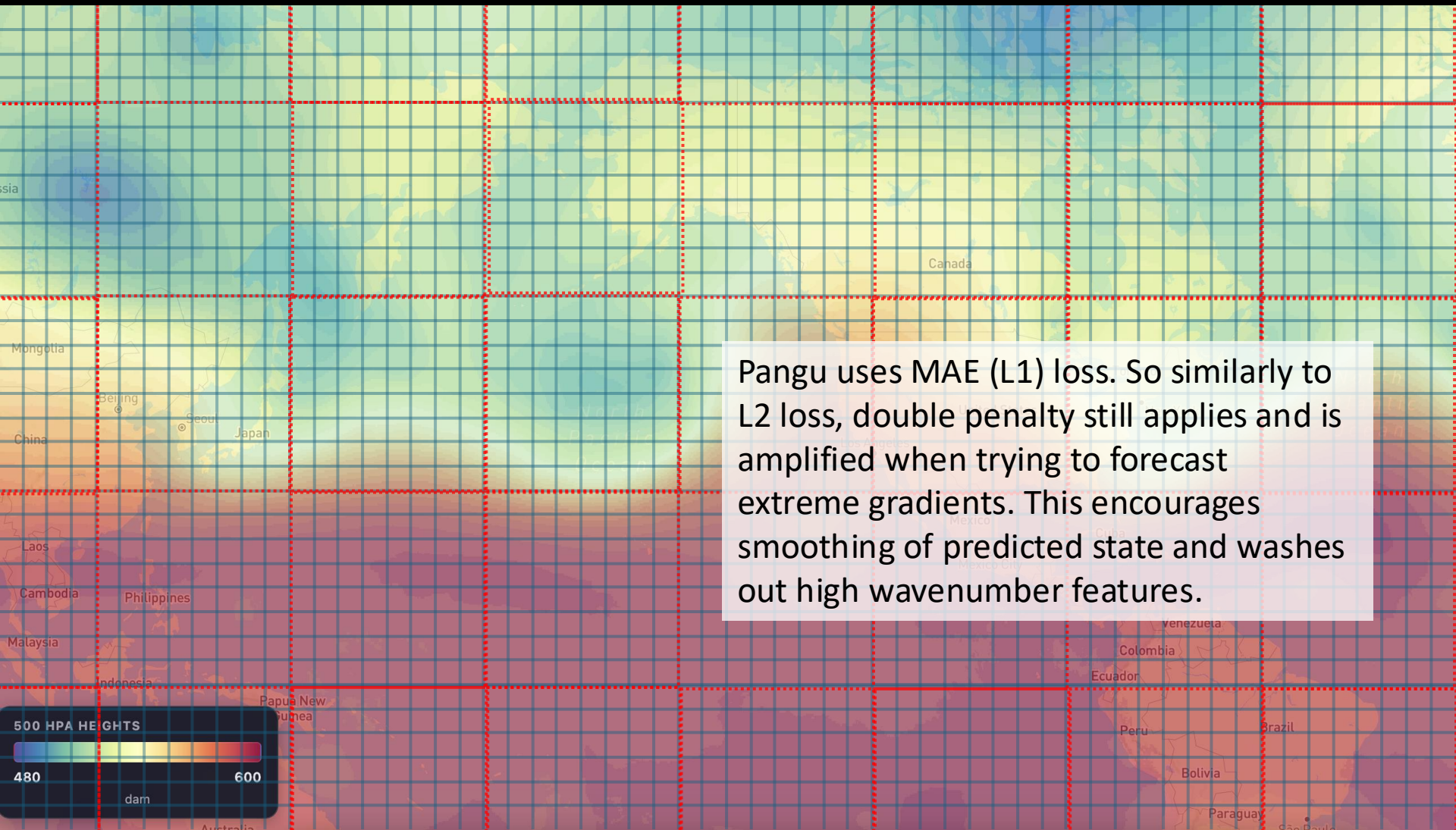
$$MLP = W_2 \cdot GELU(W_1 z_{att} + b_1) + b_2$$

MLP is like a mini FCNN in each window. It updates the state for each token. This is where "learned" physics are encoded. GELU (activation function) introduces more nonlinearity.

<u>Attention learns where information within a window is gathered from, while MLP learns how that information changes the state of each token and by how much.</u>

In the next block, shift windows by half a width in both directions and repeat. This is the underlying mechanisms of Swin transformers.

Tokens can now "talk" to tokens that were previously in other windows, enabling gradual transfer of information across the globe.

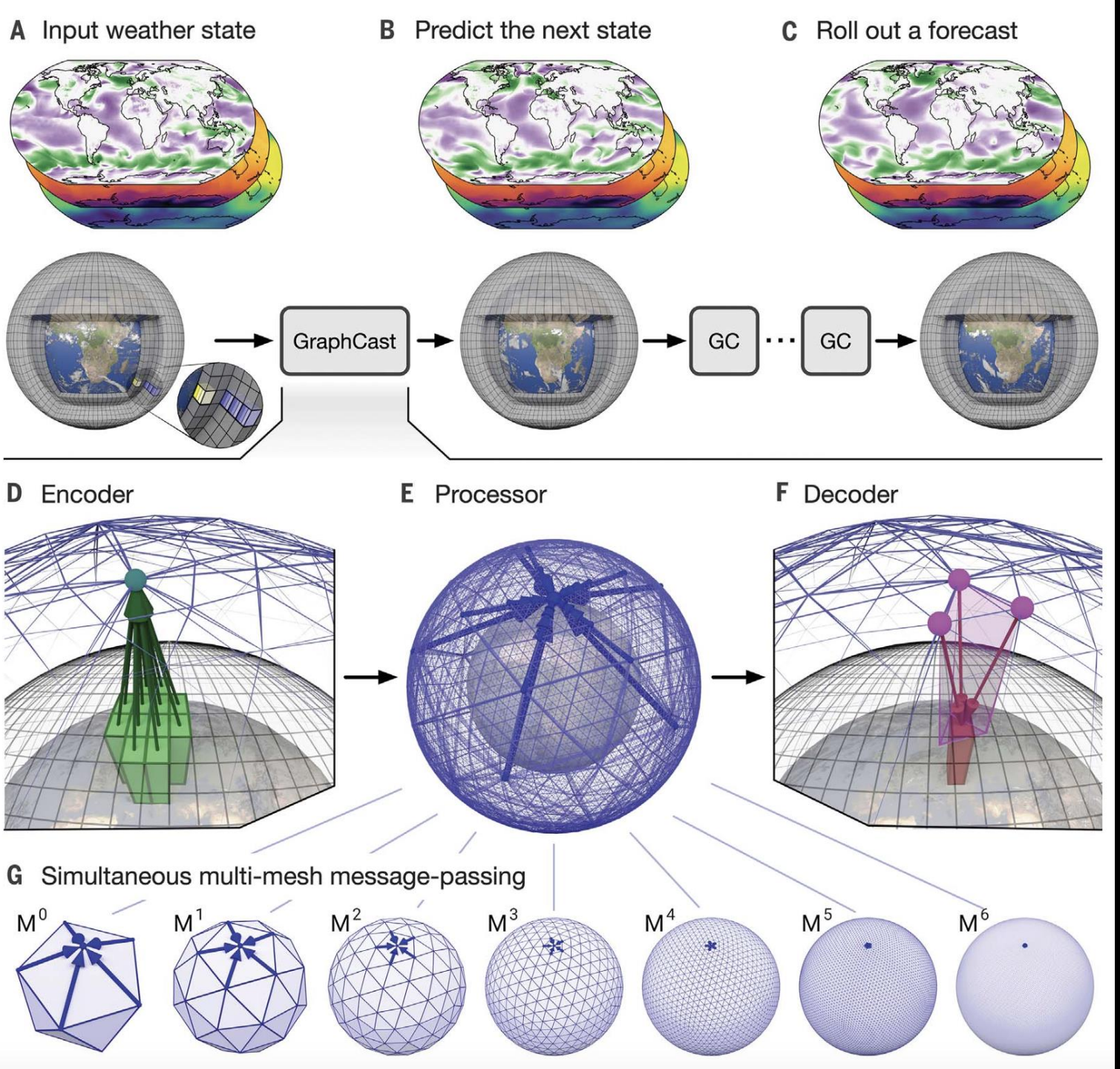


Pangu uses MAE (L1) loss. So similarly to L2 loss, double penalty still applies and is amplified when trying to forecast extreme gradients. This encourages smoothing of predicted state and washes out high wavenumber features.

Graph Neural Networks

Lam et al. (2023)

GraphCast



Information Sharing in Pangu vs. GraphCast

Both Pangu and GraphCast share information over long distances, enabling both small-scale and large-scale connections between grid points. However, the way information is passed varies significantly.

	Pangu	**GraphCast**
ML Architecture	Swin Transformer	Graph Neural Network
Local information sharing	Shared attention within window	Explicit edges on fine grid
Synoptic to global scale information sharing	Shifted windows transfer data gradually	Explicit edges on coarse grid
Nonlinearity	Embedded (e.g., softmax) and learned (weights with GELU)	Learned (weights with “swish”, similar to GELU)
Loss	MAE weighted toward upper-air	MSE weighted toward surface

$$\mathcal{L}_{\text{MSE}} = \underbrace{\frac{1}{|D_{\text{batch}}|} \sum_{d_0 \in D_{\text{batch}}}}_{\text{forecast date-time}} \underbrace{\boxed{\frac{1}{T_{\text{train}}} \sum_{\tau \in 1:T_{\text{train}}}}}_{\text{lead time}} \underbrace{\frac{1}{|G_{0.25^\circ}|} \sum_{i \in G_{0.25^\circ}}}_{\text{spatial location}} \underbrace{\sum_{j \in J}}_{\text{variable-level}} s_j w_j a_i \underbrace{(\hat{x}_{i,j}^{d_0+\tau} - x_{i,j}^{d_0+\tau})^2}_{\text{squared error}} \quad (19)$$

rollout

where

- $\tau \in 1 : T_{\text{train}}$ are the lead times that correspond to the T_{train} autoregressive steps.
- $d_0 \in D_{\text{batch}}$ represent forecast initialization date-times in a batch of forecasts in the training set,
- $j \in J$ indexes the variable, and for atmospheric variables the pressure level. E.g. $J = \{\text{Z1000}, \text{Z850}, \ldots, \text{2T}, \text{MSL}\}$,
- $i \in G_{0.25^\circ}$ are the location (latitude and longitude) coordinates in the grid,
- $\hat{x}_{i,j}^{d_0+\tau}$ and $x_{i,j}^{d_0+\tau}$ are predicted and target values for some variable-level, location, and lead time,
- s_j is the per-variable-level inverse variance of time differences,
- w_j is the per-variable-level loss weight,
- a_i is the area of the latitude-longitude grid cell, which varies with latitude, and is normalized to unit mean over the grid.

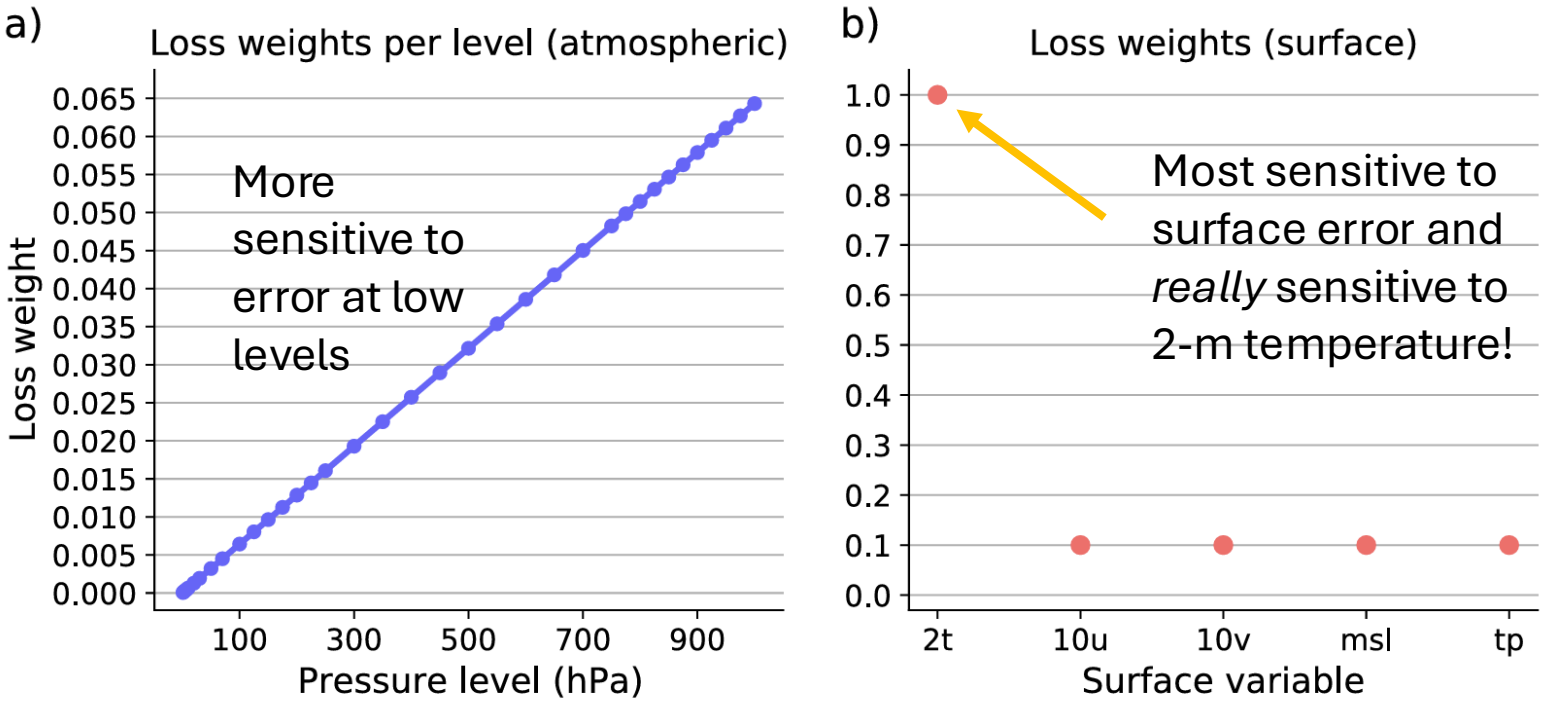
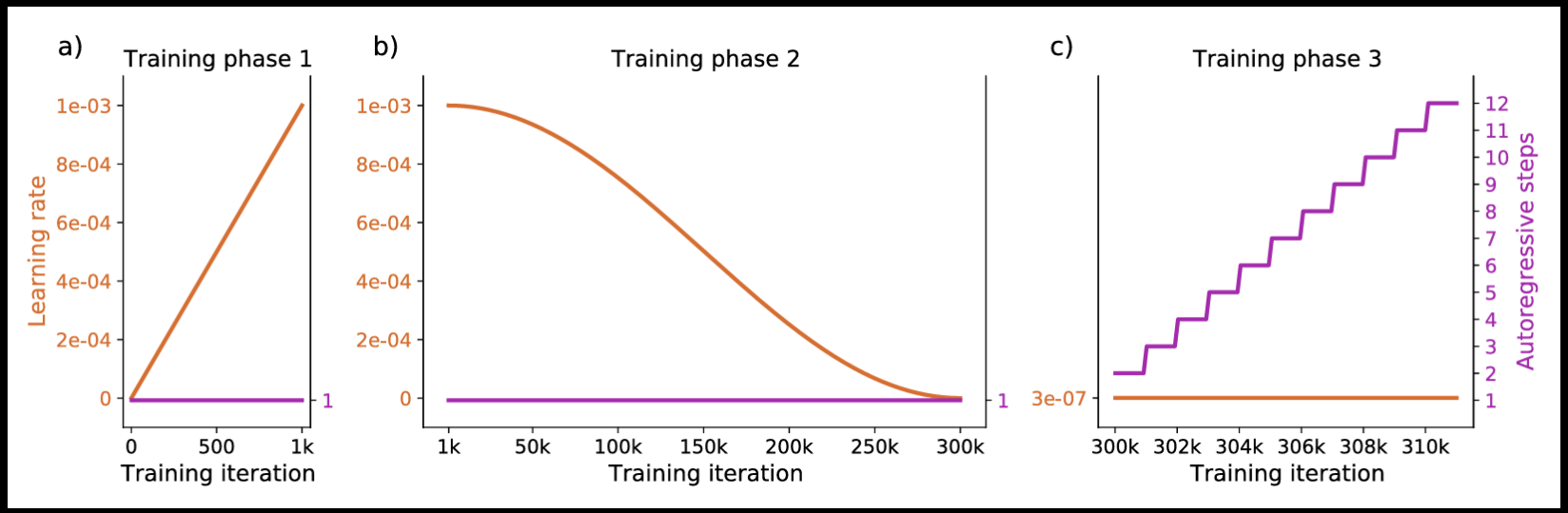


Fig. S2: **Training loss weights.** (a) Loss weights per pressure level, for atmospheric variables. (b) Loss weights for surface variables.

Error Backpropagation with “Curriculum Learning”

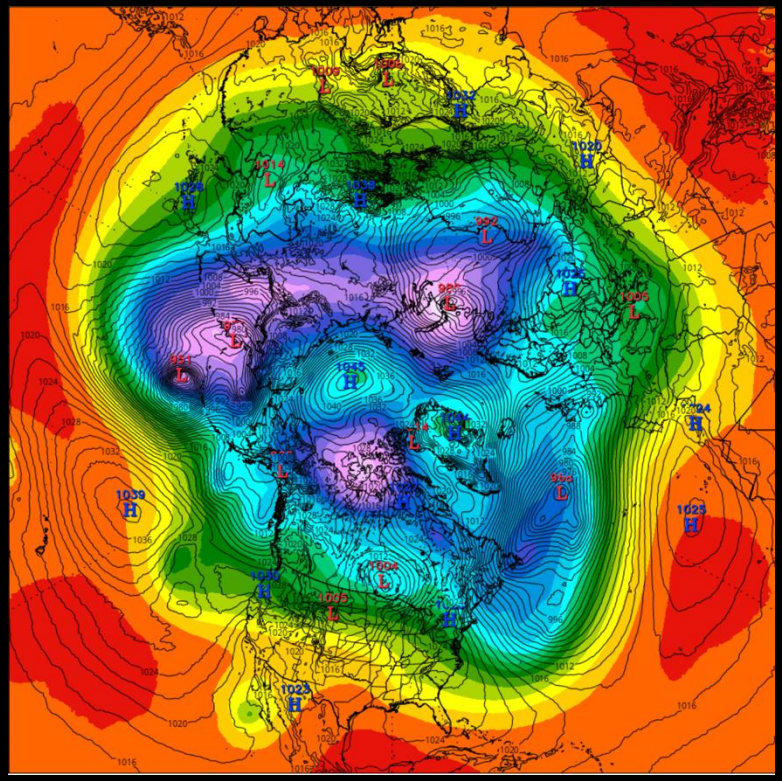
During inference, a model must take its own imperfect forecast then make a prediction based on that.

GraphCast attempts to fix this using **autoregressive rollout** to learn where errors drive the largest model instability. Weights are adjusted accordingly.



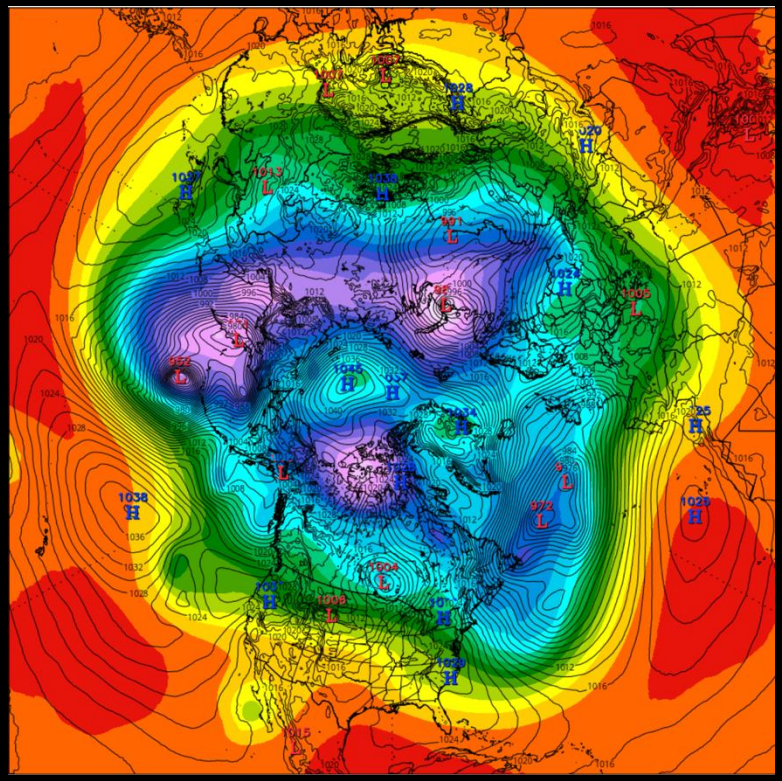
Using GNNs in Operational Forecast Centers

AIGFS



13-level GDAS → ERA5

AIFS



13-level IFS analysis → ERA5

Plots from Tropical Tidbits